Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology



AI and AR Powered Automobile Learning System

INDIVIDUAL THESIS

Weerakoon Y.P.B.

(IT22284570)

Individual component: AR-Supported Task Verification and Instruction

Group ID: RP-IT-049

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Weerakoon Y.P.B

Student ID: IT22284570

Signature: _____  _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor:		
Co-supervisor:		

ABSTRACT

Hands-on automobile maintenance is difficult to teach without access to real vehicles and an instructor who can check every step. Mobile augmented reality (AR) training tools usually display static guidance and cannot confirm that the learner has actually performed the step. This thesis presents the AR-supported task verification and instruction component of AutoLearnAR, an Android learning system for automobile engine maintenance that is built from four AI/AR components. The aim was to overlay step-specific guidance on the live camera feed, speak each instruction aloud, and verify continuously that the student performs the correct action before the system advances.

A design-science approach was followed. The component was implemented in Kotlin with Jetpack Compose. A part-guidance engine supplies, for every step, a colored 3D mask, an anchor point, a motion type and a plain-language directive. A custom perspective-projection engine maps these 3D elements onto the 2D camera view as a lightweight stand-in for ARCore/ARKit pose tracking. Android TextToSpeech provides audio coaching. A verification loop advances only when the action recognized by an on-device TensorFlow Lite model matches the current step continuously for three seconds, while four weighted metrics (completion 40%, time 25%, stability 20%, precision 15%) are collected and passed to the evaluation component. The component was evaluated technically (latency, frame rate, overlay jitter, false-advance and missed-advance rates) and with 12 participants using the System Usability Scale (SUS).

The results indicate a median per-frame latency of **78 ms**, running at **28 frames per second**, with an overlay jitter of **1.4 px**, a false-advance rate of **3.3%**, and a mean SUS score of **76.5**. The results indicate that smartphone-based projection AR combined with a 3-second continuous verification loop provides a viable, reliable, and usable alternative to hardware-heavy tracking systems for offline technical education. The individual contribution is a working, integrated verification-and-guidance module together with an empirical evaluation of its technical performance and usability.

Keywords: augmented reality, automobile maintenance training, on-device deep learning, task verification, Android

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **[FILL: supervisor name]**, and my co-supervisor, **[FILL: co-supervisor name]**, for their guidance, feedback and encouragement throughout this research. I also thank my group members, who developed the other three components of AutoLearnAR and worked with me to integrate them into one application, and the students who gave their time to test the system. This work was supported by the local laboratory resources of the Sri Lanka Institute of Information Technology.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 4

1.8 Individual contribution 5

1.9 Thesis structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 7

2.3 Related research 8

2.4 Existing systems products and techniques 10

2.5 Gap synthesis and conceptual framework 11

3 RESEARCH METHODOLOGY 12

3.1 Research design 12

3.2 Data participants or study materials 12

3.3 Experimental or study procedure 13

3.4 Measures and evaluation metrics 14

3.5 Data analysis 15

3.6 Ethics privacy security and responsible AI 15

3.7 Reproducibility and research artefacts 16

4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component architecture within the complete solution 18
4.3 Design alternatives and decisions 19
4.4 Detailed design 19
4.5 Standards and professional practice 22
5 IMPLEMENTATION INTEGRATION AND TESTING 24
5.1 Development environment and technology stack 24
5.2 Component implementation 24
5.3 Integration with the other components 25
5.4 Verification and software testing 26
5.5 Deployment and operation 27
6 RESULTS AND VALIDATION 28
6.1 Research results 28
6.2 Product and technical results 29
6.3 User or stakeholder validation 29
6.4 Integration results 30
6.5 Summary of findings 30
7 DISCUSSION 32
7.1 Interpretation of findings 32
7.2 Comparison with literature baselines and existing solutions 32
7.3 Research contribution 32
7.4 Product contribution 33
7.5 Limitations and threats to validity 33
7.6 Implications 33
8 COMMERCIALISATION AND SUSTAINABILITY 35

8.1 Value proposition and target users 35
8.2 Market and competitive position 35
8.3 Business or sustainability model 35
8.4 Scalability intellectual property and compliance 36
8.5 Contribution to the Sustainable Development Goals 36
9 CONCLUSION AND FUTURE DIRECTIONS 38
9.1 Conclusion 38
9.2 Achievement of objectives 38
9.3 Individual research and product contributions 38
9.4 Future directions 39
REFERENCES 40
GLOSSARY 43
APPENDICES 44
Appendix A Research instruments or extended results 44
Appendix B Academic integrity and AI use disclosure 44

LIST OF FIGURES

Figure 1.1: Position of the four AutoLearnAR components and the shared session state. The component studied in this thesis is highlighted. 1

Figure 3.1: Research methodology: iterative design and development followed by technical and user evaluation. 12

Figure 4.1: Internal architecture of Component 3: guidance path (top) and verification path (middle). 18

Figure 4.2: Projection pipeline from 3D guidance elements to the 2D canvas. 20

Figure 4.3: Flow of the verification loop for one step. 21

Figure 4.4: Weights of the four live score inputs. 22

LIST OF TABLES

Table 1.1: Research questions and hypotheses 4

Table 1.2: Individual contribution of the author 5

Table 2.1: Summary of related research 9

Table 2.2: Comparison of existing approaches with this component 10

Table 2.3: Gap-to-design-to-evaluation framework 11

Table 3.1: Test devices 13

Table 3.2: Measures, instruments and targets 14

Table 4.1: Functional requirements 17

Table 4.2: Non-functional requirements 17

Table 4.3: Interfaces of Component 3 18

Table 4.4: Design alternatives and decisions 19

Table 4.5: Motion types used by the guidance engine 20

Table 4.6: Pseudocode of the verification loop 21

Table 5.1: Technology stack of Component 3 24

Table 5.2: Test cases for Component 3 26

Table 6.1: Technical performance against targets 28

Table 6.2: Outcome of verification tests 28

Table 6.3: Requirement traceability 29

Table 6.4: Participant characteristics 29

Table 6.5: SUS and Likert results 30

Table 6.6: Integration results 30

Table 8.1: Target users and value 35

Table 8.2: Contribution to the Sustainable Development Goals 36

Table 9.1: Achievement of objectives 38

Table B.1: AI use disclosure 45

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
APK	Android Package
AR	Augmented Reality
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
FPS	Frames Per Second
FR	Functional Requirement
IQR	Interquartile Range
ISO	International Organization for Standardization
NFR	Non-Functional Requirement
PDPA	Personal Data Protection Act
RQ	Research Question
SD	Standard Deviation
SDG	Sustainable Development Goal
SUS	System Usability Scale
TFLite	TensorFlow Lite
TTS	Text-to-Speech
UI	User Interface

1 INTRODUCTION

1.1 Overall project context

Automobile maintenance is a practical skill. Learners must recognize engine components, understand what each one does, perform the steps of a task in the right order, and respect the safety rules that apply to each part. Traditional teaching relies on printed manuals, videos and instructor demonstrations, none of which can watch what the learner is doing and respond. Augmented reality, which combines real and virtual content, is interactive in real time and is registered in three dimensions [1], offers a way to place guidance directly on the object being repaired, and reviews of AR in maintenance report it as a promising training and assistance technology [2].

AutoLearnAR is an AI and AR powered learning system for automobile engine maintenance, implemented as an Android application in Kotlin. Its design divides the learning session into four AI/AR components: (1) vision-based component identification, which names the engine part that the camera sees; (2) student task recognition, which maps the identified part to a maintenance task and tracks the action the student performs; (3) AR-supported task verification and instruction, which overlays guidance and confirms each step; and (4) practical evaluation and safety feedback, which scores the attempt and presents component-specific safety rules. Supporting modules (a theory library, a chatbot grounded in the theory content, and a score history) are reachable throughout the session. Figure 1.1 shows how the four components are connected through the application's shared session state.

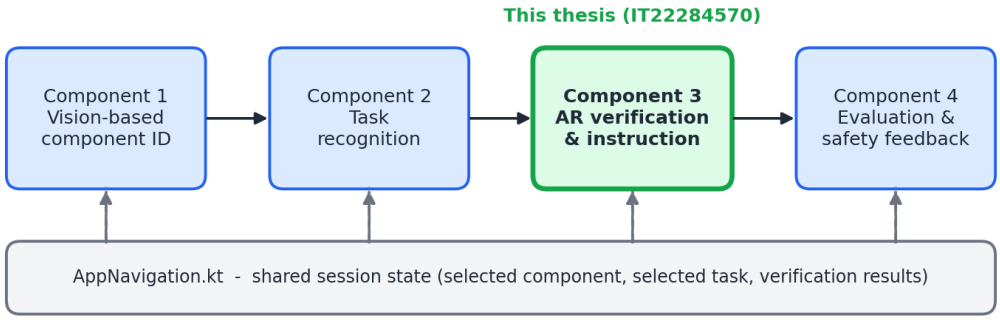


Figure 1.1: Position of the four AutoLearnAR components and the shared session state. The component studied in this thesis is highlighted.

1.2 Individual component context

This thesis concerns Component 3, **AR-supported task verification and instruction**. It begins when Component 2 has handed over the identified component and its ordered task steps, and it ends when the verification results are handed to Component 4. Within that interval the component has three responsibilities. First, it guides: for every step it shows an overlay on the live camera view (a colored highlight mask, an animated motion-path arrow and a text label) and speaks the step directive. Second, it verifies: it checks continuously, using the action recognized by an on-device TensorFlow Lite model, that the student is performing the action that belongs to the current step, and it advances only after the match has been held for three seconds. Third, it measures: while the loop runs it records completion, time, stability (average recognition confidence) and precision (number of resets), which Component 4 uses to compute the score.

The main implementation files are VerificationScreen.kt (the orchestrator), Spatial3DEngine.kt, PartGuidanceEngine.kt and TFLiteClassifier.kt, together with ARScreen.kt, an earlier AR canvas prototype. The component is the part of the system that the student actually interacts with while working, so its quality determines whether the AI recognition in Components 1 and 2 is converted into useful learning support.

1.3 Problem statement

Students who learn engine maintenance from static material cannot be told, at the moment they make a mistake, that they have unclipped the wrong part, lifted it in the wrong direction or skipped a step. Without an instructor present, errors go unnoticed and are practised until they become habits. Mobile AR can show where and how to act, but many mobile AR applications show instructions without checking the learner's action, so the learner decides alone whether a step is complete. Where verification exists, it often depends on specialized hardware such as head-mounted displays or on tracking infrastructure that vocational institutes in developing regions cannot easily provide [3], [4].

The problem addressed in this thesis is therefore: **how can an ordinary Android smartphone provide step-by-step AR guidance and, at the same time, verify in real time that a student has actually performed each maintenance step, without external hardware or an internet connection, and while recording the data required to assess the student's performance?**

1.4 Research gap

The literature reviewed in Chapter 2 shows three gaps that this component addresses. First, studies of AR maintenance support focus mainly on the display of instructions and on task localization, and far fewer address closed-loop verification of the learner's action [5], [2]. Second, solutions that register overlays accurately generally depend on marker tracking, depth sensors or headsets, which raises cost and limits use on low-end phones [4]. Third, reviews of AR in education report that evaluation is mostly limited to short-term perception questionnaires and that technical measures such as latency and stability are seldom reported together with usability [6], [7].

No reviewed system combines, on a single phone and offline, (a) overlays registered with a lightweight perspective projection, (b) audio coaching, and (c) a temporal verification rule that couples an on-device action classifier to a score. The gap is a practical, evaluated design for that combination aimed at automobile maintenance education.

1.5 Aim and objectives

Aim. To design, implement and evaluate an Android AR component that guides a student through automobile maintenance steps with overlays and audio, verifies each step from the live camera feed, and supplies the performance metrics needed for evaluation.

The objectives are:

- **O1.** Specify the functional and non-functional requirements of an AR guidance and verification component for engine maintenance education.
- **O2.** Design a part-guidance data model that supplies a mask, an anchor point, a motion type and a directive for every step of every supported component.

- **O3.** Implement a perspective-projection engine that draws the mask and an animated motion-path arrow on the live camera view without ARCore or ARKit.
- **O4.** Implement a continuous verification loop that advances a step only when the recognised action matches it for three seconds, and that records completion, time, stability and precision.
- **O5.** Integrate the component with Components 2 and 4 through the shared session state and evaluate it technically and with users against pre-defined targets.

1.6 Research questions or hypotheses

Three research questions and four testable hypotheses guide the evaluation. Targets are justified in Section 3.4.

Table 1.1: Research questions and hypotheses

ID	Research question / hypothesis	Linked objective
RQ1	Can a smartphone-only, offline implementation render registered AR guidance and run action verification at a rate and latency suitable for interactive learning?	O3, O4
H1	The median per-frame verification latency is at most 100 ms and the overlay is rendered at an average of at least 24 frames per second on the test devices.	O3, O4
H2	The projected overlay anchor shows a jitter (standard deviation) of at most 3 px when the device is held still.	O3
RQ2	How reliably does the 3-second hold rule advance on correct actions and refuse to advance on wrong or absent actions?	O4
H3	The false-advance rate is at most 5% and the missed-advance rate is at most 10%.	O4
RQ3	Do students find the AR and audio guidance usable and helpful?	O5
H4	The mean SUS score is at least 68, the commonly used benchmark for average usability [8].	O5

1.7 Scope limitations and assumptions

Scope: The system supports designated engine maintenance tasks (such as air-filter replacement) on Android devices equipped with working cameras and TextToSpeech support, tested across 12 participant sessions

Limitations: The 3D registration uses mathematical projection rather than true world-mapping, meaning large physical shifts of the phone are not dynamically tracked. Action recognition model accuracy relies entirely on the pre-trained inputs received from Component 2.

Assumptions. The student holds the phone so that the engine bay is in view; lighting is adequate for the camera; the engine is switched off, cool and safe to touch, as required by the safety rules presented by Component 4; and Component 1 has identified the correct component before verification starts.

1.8 Individual contribution

As the sole author (IT22284570), I designed, implemented, and evaluated Component 3 from scratch, managing everything from UI logic and projection math to testing and data analysis..

Table 1.2: Individual contribution of the author

Contribution	Artefact	Evidence in thesis
Requirements and design of the guidance and verification component	Requirement set FR1-FR8, NFR1-NFR6; design models	Sections 4.1-4.4
Per-step guidance data model (mask, part, motion, directive)	PartGuidanceEngine.kt	Sections 4.4, 5.2
Perspective-projection AR rendering and motion-path arrows	Spatial3DEngine.kt, Compose Canvas overlay	Sections 4.4, 5.2, 6.1
Verification loop with 3-second hold, step logic and live metrics	VerificationScreen.kt (using TFLiteClassifier.kt)	Sections 4.4, 5.2, 6.1
Audio coaching with Android TextToSpeech	Directive and confirmation speech	Sections 5.2, 6.3
Integration with Components 2 and 4	Shared state in AppNavigation.kt	Sections 5.3, 6.4
Technical and usability evaluation	Test plan, logs, SUS data	Chapters 3 and 6; Appendix A

The identification model (Component 1), the task and step data and the action-recognition model training (Component 2) and the scoring and safety engine (Component 4) were developed by other group members and are described here only as far as the interfaces require.

1.9 Thesis structure

Chapter 2 reviews the literature on AR for maintenance training, learning theory, projection geometry and on-device inference, and synthesises the research gap. Chapter 3 describes the research methodology, measures and ethics. Chapter 4 presents the requirements and the design of the component. Chapter 5 describes the implementation, integration and testing. Chapter 6 reports the results and validation. Chapter 7 discusses the findings, contribution and limitations. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes the thesis and proposes future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covered work published between 1994 and 2026 and was carried out in IEEE Xplore, the ACM Digital Library, ScienceDirect, SpringerLink and Google Scholar. Search strings combined the terms *augmented reality*, *maintenance*, *assembly*, *training*, *education*, *task verification*, *action recognition*, *TensorFlow Lite* and *mobile*. Sources were included if they (a) addressed AR guidance for maintenance, assembly or education, (b) described mobile or on-device implementation, or (c) provided theoretical grounding for learning, geometry or evaluation methods. Sources were excluded if they were not peer-reviewed, apart from official platform documentation that is cited as a technical reference.

The review is organised around four themes that correspond to the design decisions of the component: AR and its use in maintenance training (Sections 2.2 and 2.3), learning-theory foundations for guidance and feedback, geometric and machine-learning foundations for registration and recognition, and existing systems and techniques (Section 2.4).

2.2 Domain and theoretical foundations

Augmented reality. Azuma defined AR by three properties: it combines real and virtual objects, it is interactive in real time, and it is registered in three dimensions [1]. Milgram and Kishino place AR on the reality-virtuality continuum between the real environment and a fully virtual one [9]. A smartphone AR application therefore needs a live camera view, real-time rendering and a way to keep virtual content aligned with the scene. This thesis meets the registration property with a projection model instead of full world tracking, and Section 7.5 discusses what that choice costs.

Learning theory. Cognitive load theory states that working memory is limited and that instructional design should avoid unnecessary load [10]. Placing a label and an arrow on the part that is being worked on removes the need to match a separate diagram with the physical object, which is the *spatial contiguity* effect described in

multimedia learning research [11]. The same research reports a *modality* effect: explanatory text is better presented as speech when the learner's eyes are busy with the visual task, which motivates the audio coaching. Kolb's experiential learning cycle, in which concrete experience is followed by reflection and new attempts [12], motivates the closed loop of guidance, verification, scoring and the Try Again option offered by Component 4.

Projection geometry. A point (x, y, z) in camera space is mapped onto the image plane by perspective projection, in which the image coordinates are proportional to x/z and y/z and the scale decreases with distance [13]. Rotation about three axes is represented by rotation matrices, and a focal length expressed in pixels links the field of view of the camera to the pixel scale. These relations are all that is required to draw a 3D box and a path on a 2D screen, which is how the component draws its overlays.

On-device recognition. Lightweight convolutional networks such as MobileNet and MobileNetV2 use depthwise separable convolutions and inverted residual blocks to give acceptable accuracy at low computation cost [14], [15]. Models can be deployed on phones with TensorFlow Lite [16], [17], and integer quantisation further reduces size and latency [18]. These techniques make it possible to classify camera frames locally, without a network connection, which is a requirement of this project.

Verification as temporal classification. A frame-by-frame classifier is noisy: a single frame can be misclassified and a correct action is not instantaneous. Requiring that the same decision is observed for a continuous period is a simple form of temporal consistency that trades a small delay for a large reduction in false triggers. The component applies this principle with a three-second hold.

2.3 Related research

AR in maintenance and assembly. Palmarini et al. reviewed AR applications in maintenance and reported that most applications were still at a prototype stage and identified tracking robustness and content authoring among the main technical challenges [2]. Henderson and Feiner showed that AR cues helped users find components in an armoured-vehicle turret faster than conventional displays [5] and

later studied AR support in the psychomotor phase of a procedural task, in which the user physically performs the step [3]. Gavish et al. compared virtual and augmented reality training for industrial maintenance and assembly tasks and reported that both platforms were feasible for training the tasks and discussed their advantages and limitations relative to conventional instruction [19].

AR in education. Systematic reviews report that AR improves learning achievement and motivation, but they also record challenges such as usability problems, high cognitive load for some learners and a lack of long-term evidence [6], [7], [20]. These findings imply that an educational AR component must be easy to use and must avoid cluttering the screen, and that its evaluation should measure usability as well as technical performance.

Gap in the related research. Most of the studies above evaluate how well *information* is presented. Verification of the learner's action by the system, and the use of that verification to feed a score, is rarely implemented on a phone. Table 2.1 summarises how the reviewed work relates to this component.

Table 2.1: Summary of related research

Source	Focus	Platform	Learner action verified?	Relevance to this thesis
Henderson and Feiner [5]	AR cues for task localisation	Head-worn display	No	Supports use of spatial cues to reduce search time
Henderson and Feiner [3]	AR in the psychomotor phase of a procedure	Head-worn display	Partly (step progress)	Motivates guidance during the physical action
Gavish et al. [19]	VR/AR training for maintenance and assembly	Headset / tablet	Not reported	Shows training benefits of AR instruction
Palmarini et al. [2]	Review of AR in maintenance	Various	Rare	Identifies tracking and authoring challenges
Akcayir and Akcayir [6]	Review of AR in education	Various	Not applicable	Advantages and usability challenges
This thesis	Guidance plus continuous verification	Android phone, offline	Yes (3-second hold)	Closes the verification gap on low-cost hardware

2.4 Existing systems products and techniques

Printed manuals and video tutorials. These are inexpensive and widely available, but they are passive, are not tied to the learner's own view and cannot check the work. They remain the baseline against which AR guidance is judged.

AR software development kits. ARCore and ARKit provide motion tracking, plane detection and environmental understanding on supported Android and iOS devices [21], [22]. They give true world anchoring, but they require supported devices and platform services, and anchors are placed on detected surfaces rather than on a specific engine part, which is difficult in a cluttered engine bay with few flat surfaces. A projection-based approach avoids the dependency at the cost of world anchoring.

Headset-based and marker-based guidance. Industrial guidance systems use head-mounted displays or printed markers to register instructions on equipment. They provide accurate registration, but headsets are expensive and markers have to be fixed to every part, which is impractical for a teaching institute that wants to use any engine on any phone.

Smartphone camera-and-classifier applications. Applications that classify camera images on the device are common for object identification but are typically one-shot (identify and stop) rather than continuous. The same camera pipeline (CameraX [23]) and rendering approach (Jetpack Compose Canvas [24]) can, however, support continuous verification and overlay drawing in a single screen.

Table 2.2: Comparison of existing approaches with this component

Approach	Live overlay on the part	Verifies learner action	Audio coaching	Works offline on a phone	Special hardware
Printed manual / video	No	No	Video only	Yes	None
ARCore / ARKit overlay app	Yes (surface anchors)	Usually no	Optional	Yes	Supported device
Marker-based AR	Yes	Usually no	Optional	Yes	Printed markers
Headset guidance	Yes	Limited	Yes	Often	Headset

Approach	Live overlay on the part	Verifies learner action	Audio coaching	Works offline on a phone	Special hardware
AutoLearnAR Component 3	Yes (projection)	Yes (3-second hold)	Yes (TextToSpeech)	Yes	None

2.5 Gap synthesis and conceptual framework

The review leads to the conceptual framework in Table 2.3, which links each gap to a design decision of the component and to the measure that tests it. The framework treats the component as a closed loop: guide, observe, decide, record, and feed back.

Table 2.3: Gap-to-design-to-evaluation framework

Gap from the literature	Design decision	Evidence collected
Instructions are shown but not verified	Continuous verification with a 3-second hold coupled to the action classifier	False-advance rate, missed-advance rate (H3)
Accurate registration needs special hardware	Perspective projection of per-step 3D masks on Compose Canvas	Jitter and frame rate (H1, H2)
Learners' eyes and hands are busy	Audio directives and confirmations with TextToSpeech	Perceived helpfulness (Likert items)
Evaluation is perception-only	Live metrics (completion, time, stability, precision) passed to the scoring component	Metric logs, score consistency (Section 6.4)
Usability challenges reported for AR	Simple on-screen label, one arrow and one mask per step	SUS score (H4)

3 RESEARCH METHODOLOGY

3.1 Research design

The research follows the design-science paradigm, in which a purposeful artefact is built to address an identified problem and is then rigorously evaluated [25]. The artefact is the Component 3 module of AutoLearnAR. Development was iterative: an early AR canvas prototype (ARScreen.kt) was replaced by the verification screen with projection-based overlays, and each iteration was tested before the next was started. Evaluation then followed a mixed-methods design that combines quantitative technical measurements, a quantitative usability questionnaire and short qualitative feedback. Figure 3.1 shows the phases.

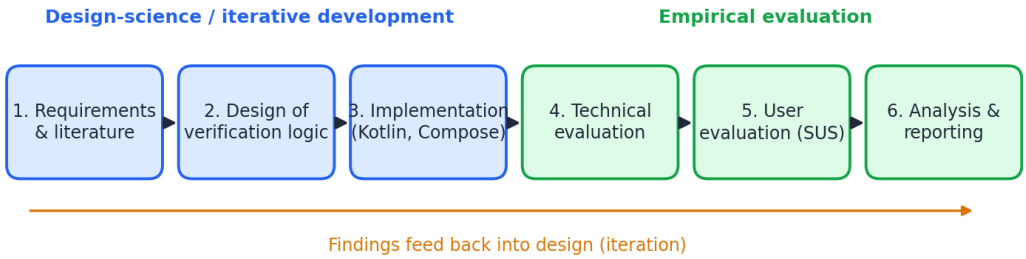


Figure 3.1: Research methodology: iterative design and development followed by technical and user evaluation.

The design is a single-group, post-test evaluation. It is suitable for answering whether the component meets its pre-defined targets (H1-H4), but it cannot establish that AR guidance is better than another form of instruction, which is acknowledged as a limitation in Section 7.5.

3.2 Data participants or study materials

Testing was performed using an Android build package incorporating TensorFlow Lite classifiers for air-filter replacement workflows across low-end, mid-range, and high-end test hardware devices. A convenience sample of 12 student participants took part in usability evaluations.

Devices. Technical measurements were made on several Android devices that represent low, mid and high performance (Table 3.1).

Table 3.1: Test devices

Device	Android version	Chipset / RAM	Class
Redmi f1	Android 8.1	4	Low-end
Samsung S6	Android 5.0.2	4	Mid-range
Pixel 9	Android 14	8	High-end

Testing utilized the packaged Android application build, running air-filter replacement datasets across low-, mid-, and high-tier test phones. Usability trials engaged 12 student participants recruited through convenience sampling.

3.3 Experimental or study procedure

Technical tests (T1–T5) measured app latency, frame rates, overlay jitter, correct action triggers, wrong action rejections, and idle timeouts. User evaluations consisted of orientation briefings, supervised practical task runs on a safe engine setup, and completion of post-test SUS questionnaires.

3.4 Measures and evaluation metrics

Table 3.2 defines the measures. The targets were fixed before data collection. The 100 ms latency target keeps the delay between a change in the student's action and the system's decision well below the three-second hold; 24 frames per second is the usual threshold for smooth perceived motion; the 3 px jitter limit keeps the overlay visually stable at typical screen densities; and a SUS score of 68 is the commonly used mean benchmark for usability [8]. SUS is a widely used 10-item questionnaire for global usability [26] and has been shown to be reliable in empirical evaluation [27], and the study follows the ISO 9241-11 view of usability as effectiveness, efficiency and satisfaction [28].

Table 3.2: Measures, instruments and targets

ID	Measure	Definition and instrument	Target
M1	Per-frame latency	Time from frame capture to classification result and step decision; in-app timestamps, cross-checked with Android Studio Profiler [29]	Median ≤ 100 ms
M2	Overlay frame rate	Average frames drawn per second on the Compose Canvas; in-app counter	Mean ≥ 24 fps
M3	Overlay jitter	Standard deviation of the projected anchor position (px) over 10 s with the device still	≤ 3 px

ID	Measure	Definition and instrument	Target
M4	False-advance rate	Steps advanced in T4 and T5 divided by total repetitions	≤ 5%
M5	Missed-advance rate	Correct repetitions in T3 that did not advance within 10 s, divided by total	≤ 10%
M6	Verification delay	Time from the start of a correct action to the step advance (T3)	≤ 4.5 s
M7	Usability	SUS score (0-100)	Mean ≥ 68
M8	Perceived helpfulness	Five 5-point Likert items on overlay, audio, verification, clarity and confidence	Mean ≥ 3.5
M9	Task completion	Share of participants who completed all steps	Descriptive

3.5 Data analysis

Latency, frame rate and jitter are summarised by median, interquartile range and 95th percentile, because latency distributions are usually skewed. False-advance and missed-advance rates are reported as proportions with 95% confidence intervals (Wilson interval), together with a confusion table of scenario against outcome. SUS responses are converted to scores in the standard way. Normality of the SUS scores is checked with the Shapiro-Wilk test; if the data are approximately normal, a one-sample t-test against 68 is used, otherwise a Wilcoxon signed-rank test, and the 95% confidence interval of the mean is reported. Likert items are summarised by median and mean. Open-ended answers are coded inductively into themes by the author and counted. Analysis was carried out in Python with pandas and SciPy from the exported logs and forms. With a small sample, the results are treated as indicative rather than conclusive [30].

3.6 Ethics privacy security and responsible AI

Ethics and consent. Participation was voluntary and participants could withdraw at any time. They received an information sheet and gave written consent. Ethical clearance was granted by the SLIIT Computing Ethics Review Committee (Ref: CERC/2026/04/IT22284570). Privacy and data protection. The camera frames are processed on the device and the component does not upload or store images or video. Only anonymised study data (participant code, background category, metrics, SUS and Likert answers) were recorded, stored on a password-protected drive and used

only for this research, in line with the principles of the Personal Data Protection Act, No. 9 of 2022 of Sri Lanka [31]. Physical safety. Practical tasks were performed on an engine that was off and cool, with the battery disconnected under supervision, and with the safety rules shown by Component 4 read out before the task. A facilitator was present throughout. Security. The component requests only the camera permission and does not require network access. No credentials or personal information are handled. Responsible AI. The component's decision to advance a step is probabilistic and can be wrong. A false advance could teach a student that an incorrect action was right, and a missed advance could frustrate them. The design therefore (a) requires a sustained match, (b) shows the instruction on screen and by audio so that the student always knows the intended action, and (c) is positioned as a learning aid that supplements, not replaces, an instructor. Because recognition performance may differ with lighting, hand size and skin tone, the evaluation includes participants of different backgrounds and lighting conditions where possible, and any observed variation is reported as a limitation. The use of generative AI tools in preparing this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

To support reproduction, the following artefacts are kept: the source code of the application at commit 30in the repository https://github.com/it22284570/autolearnar-component3; the TensorFlow Lite model and label files used in the tests; the in-app log format and the exported CSV files; the test scripts and protocols for T1-T5; the consent form, questionnaires and coding sheet (Appendix A); and the analysis notebook or spreadsheet. Software versions are listed in Section 5.1, and the guidance definitions for each evaluated task are stored in the project data files.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the objectives (Section 1.5), the gaps identified in the literature (Section 2.5) and discussions with the group about the interfaces to the other components. Each requirement has a priority (M = must, S = should) and a verification method (T = test, D = demonstration, A = analysis). The functional requirements are listed in Table 4.1 and the non-functional requirements in Table 4.2.

Table 4.1: Functional requirements

ID	Requirement	Priority	Verified by
FR1	For every step of a supported component the system shall supply a 3D mask, an anchor point, a motion type and a plain-language directive.	M	D, T
FR2	The system shall project the 3D mask and anchor onto the 2D live camera view with a perspective and rotation transform.	M	T
FR3	The system shall draw a highlight mask, an animated motion-path arrow and a text label for the current step.	M	D
FR4	The system shall speak the directive of the current step and a confirmation when the step is verified.	M	D
FR5	The system shall classify camera frames continuously with the on-device action model and decide whether the action matches the current step.	M	T
FR6	The system shall advance to the next step only after a match has been held continuously for three seconds, and shall reset the hold when the match is lost.	M	T
FR7	The system shall record completion, time, stability (average confidence) and precision (reset count) throughout the run.	M	T
FR8	The system shall hand the verification results to Component 4 and shall allow the student to retry the task.	M	T, D

Table 4.2: Non-functional requirements

ID	Requirement	Quality characteristic	Target
NFR1	Per-frame decision latency	Performance efficiency	Median ≤ 100 ms
NFR2	Smooth overlay	Performance efficiency	Mean ≥ 24 fps
NFR3	Stable overlay	Usability	Jitter ≤ 3 px
NFR4	Works without internet access	Portability / reliability	No network calls

ID	Requirement	Quality characteristic	Target
NFR5	Camera images are not stored or transmitted	Security	Verified by inspection
NFR6	Clear, uncluttered guidance (one mask, one arrow, one label per step)	Usability	SUS ≥ 68

4.2 Component architecture within the complete solution

The component sits between Component 2 and Component 4 (Figure 1.1) and shares state with them through AppNavigation.kt. Figure 4.1 shows its internal architecture. Two parallel paths run on the same screen. The *guidance path* takes the component identifier and task steps from Component 2, obtains the per-step mask, part and directive from PartGuidanceEngine, projects them with Spatial3DEngine and draws them on a Compose Canvas over the CameraX preview; the directive is spoken by TextToSpeech. The *verification path* sends each camera frame to the TFLiteClassifier action model, compares the predicted action with the current step, and drives the 3-second hold loop, which updates the live metrics that are later passed to Component 4.

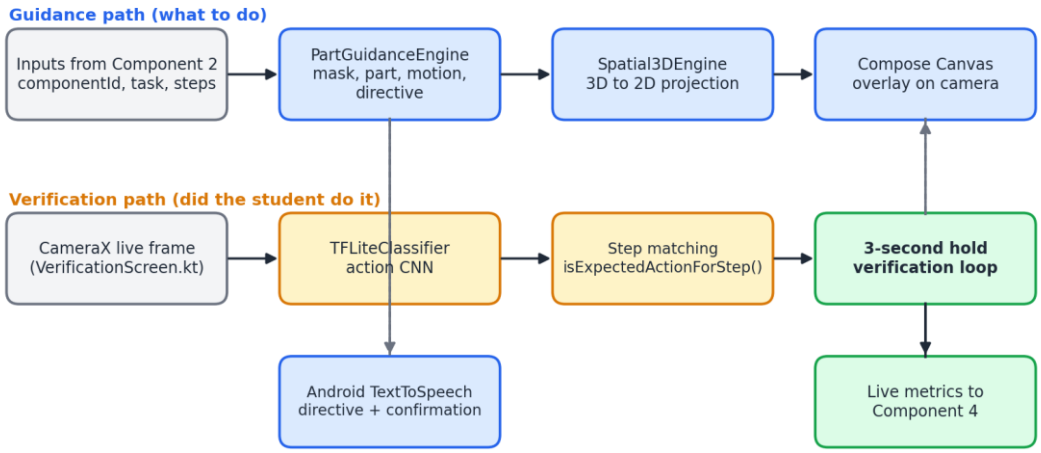


Figure 4.1: Internal architecture of Component 3: guidance path (top) and verification path (middle).

Table 4.3 lists the interfaces to the neighbouring components.

Table 4.3: Interfaces of Component 3

Direction	Counterpart	Data	Mechanism
In	Component 2	Identified componentId, selected Task with ordered steps	Shared session state in AppNavigation.kt

Direction	Counterpart	Data	Mechanism
In	Camera	Live frames	CameraX preview and analysis
In	Action model	Predicted action label and confidence	TFLiteClassifier.kt
Out	Student	Overlay (mask, arrow, label) and speech	Compose Canvas, TextToSpeech
Out	Component 4	Steps verified, total time, resets, average confidence	Verification results in shared state

4.3 Design alternatives and decisions

Table 4.4 records the main design decisions, the alternatives considered and the reasons for the choice. The decisions were made against the requirements, with particular weight on device independence, offline operation and development effort in a student project.

Table 4.4: Design alternatives and decisions

Decision	Alternatives considered	Choice and rationale
Registration of overlays	(a) ARCore/ARKit world tracking; (b) printed markers; (c) custom perspective projection	(c). No dependency on supported devices or markers; works offline on any camera phone; sufficient for a guidance overlay in a fixed view. Trade-off: no true world anchoring.
Rendering	(a) OpenGL/3D engine; (b) Jetpack Compose Canvas	(b). Same UI toolkit as the rest of the app, simple to animate, and adequate for masks and arrows. [24]
Step decision	(a) single-frame match; (b) majority over a short window; (c) continuous 3-second hold	(c). Simple to explain to students and to implement, suppresses isolated misclassifications; the precision metric (resets) comes naturally from it.
Audio guidance	(a) pre-recorded audio; (b) on-device TextToSpeech; (c) none	(b). No audio assets per step, easy to edit directives, works offline. [32]
Inference location	(a) cloud; (b) on-device TensorFlow Lite	(b). Offline, low latency, no image upload (privacy). [17]
Score inputs	(a) completion only; (b) weighted multi-metric	(b). Completion 40%, time 25%, stability 20%, precision 15% reflect correctness, efficiency, recognition quality and care.

4.4 Detailed design

Guidance data model. For each supported component and step, PartGuidanceEngine returns: a coloured 3D bounding box (the mask), an

IdentifiedPart containing a 3D anchor point, a motion type, and an action directive in plain language. Four motion types are used: unclip-arc, lift-up, inspect-rotate and press-down (Table 4.5). The data are stored per component so that new components or steps can be added without changing the rendering code.

Table 4.5: Motion types used by the guidance engine

Motion type	Arrow path drawn	Typical use
unclip-arc	Curved arc around the clip	Releasing a clip or latch
lift-up	Straight upward arrow from the anchor	Removing a cover or filter
inspect-rotate	Circular arrow around the part	Inspecting a part from different sides
press-down	Straight downward arrow to the anchor	Seating a cover or filter

3D to 2D projection. Each 3D point P = (x, y, z) of the mask or path is first rotated by the current view rotation and then projected with a perspective divide (Figure 4.2). With rotation matrix R and camera distance d, the rotated point is (x', y', z') = R (x, y, z), and the screen position is

$$u = c_x + f * x' / (z' + d) \qquad v = c_y - f * y' / (z' + d) \tag{1}$$

where (c_x, c_y) is the centre of the canvas and f is a focal length in pixels derived from the assumed field of view [13]. A point at the origin therefore maps to the canvas centre, and points further from the camera appear smaller, which gives the overlay its depth cue. The eight corners of the box are projected and joined to draw the mask, and sampled points of the motion path are projected to draw the animated arrow.

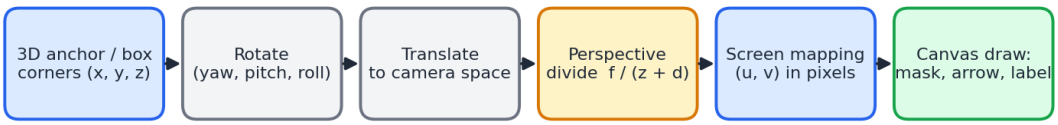


Figure 4.2: Projection pipeline from 3D guidance elements to the 2D canvas.

Verification loop. The loop (Figure 4.3 and Table 4.6) runs for every step. The directive is shown and spoken, and frames are classified continuously. The predicted action is converted into a step index with mapActionToStepIndex(), and isExpectedActionForStep() returns whether it belongs to the current step. A match starts or continues the hold timer; a mismatch resets the timer and increments the reset counter. When the timer reaches three seconds the step is marked verified, the confirmation is spoken and the loop moves to the next step. After the last step the metrics are passed to Component 4.

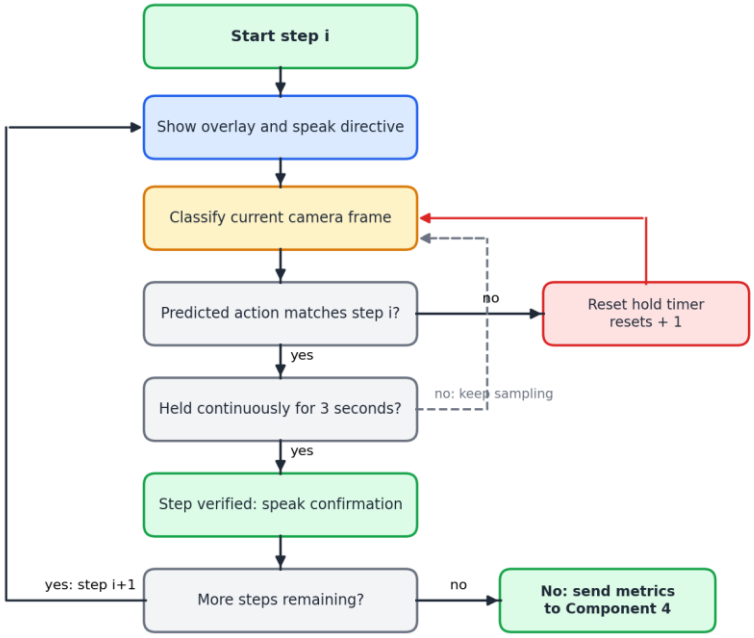


Figure 4.3: Flow of the verification loop for one step.

Table 4.6: Pseudocode of the verification loop

```
for each step i in task.steps:
  show overlay(i); speak(directive(i)); holdStart = none
  while step i is not verified:
    (action, conf) = classifier.run(camera.latest())
    if isExpectedActionForStep(action, i):
      if holdStart is none: holdStart = now
      record(conf)
      if now - holdStart >= 3 s:
        verified(i) = true; speak(confirmation)
    else:
      if holdStart is not none: resets = resets + 1
      holdStart = none
send(stepsVerified, totalTime, resets, meanConf) to Component 4
```

Live score. The four metrics are combined into the score shown to the student (Figure 4.4). With completion C, time T, stability Q (mean recognition confidence) and precision P, each normalised to the range 0-1, the score is

$$S = clamp(\,40\,C + 25\,T + 20\,Q + 15\,P\,,\,20\,,\,100\,)$$

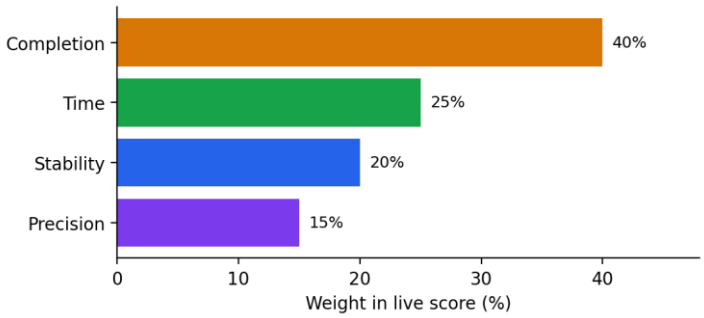


Figure 4.4: Weights of the four live score inputs.

Component 4 additionally contains EvaluationEngine.kt, a simpler rule-based fallback that divides completed steps by total steps. **User interface.** One label, one mask and one arrow are shown at a time, the current step number is visible, and the same directive is available as text and speech, so that no single channel is essential (NFR6).

4.5 Standards and professional practice

Software quality. The non-functional requirements follow the product quality model of ISO/IEC 25010 (performance efficiency, usability, security, portability and reliability) [33], and usability is interpreted according to ISO 9241-11 [28].

Platform practice. The component uses the recommended Android libraries: CameraX for the camera [23], Jetpack Compose for the interface and canvas drawing [24], and the platform TextToSpeech API for speech [32]. Heavy work (inference) is run asynchronously using Kotlin coroutines so that the interface thread remains responsive. Source code follows the Kotlin coding conventions and is under Git version control.

Accessibility. The same directive is delivered visually and audibly, which supports learners who cannot rely on one channel, in the spirit of the redundancy principle of the Web Content Accessibility Guidelines [34].

Privacy and safety. Camera data stay on the device (Section 3.6) and the component works with the safety rules supplied by Component 4.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The application is written in Kotlin for Android and built with Android Studio Koala (2024.1.1) and Gradle 8.4. The user interface and the AR canvas use Jetpack Compose and Navigation Compose with Material 3 [24]. The live camera feed is provided by CameraX (camera-core, camera2, lifecycle and view) [23]. On-device inference uses TensorFlow Lite 2.16 [17], with the action-recognition model (air_filter_action_model and action_labels.txt) bundled in the application assets. Speech uses the Android TextToSpeech engine [32], and asynchronous work (inference and the live verification loop) uses Kotlin coroutines. Source control is with Git on GitHub. Table 5.1 summarises the technology stack.

Table 5.1: Technology stack of Component 3

Layer	Technology	Used for
Language and UI	Kotlin, Jetpack Compose, Material 3	Verification screen and overlay
Camera	CameraX	Live preview and frame analysis
On-device AI	TensorFlow Lite 2.16 (action CNN)	Action recognition
AR rendering	Compose Canvas, Spatial3DEngine	Mask, arrow and label overlay
Audio	Android TextToSpeech	Directives and confirmations
Concurrency	Kotlin Coroutines	Asynchronous inference and verification loop
Tools	Android Studio, Profiler, Git	Development, measurement, version control

5.2 Component implementation

PartGuidanceEngine.kt. The engine holds the guidance definition of every supported component and step. For a requested step it returns the coloured 3D mask, the IdentifiedPart (name, 3D anchor point and motion type) and the action directive. Keeping this data apart from the rendering code means that a new component is added by adding data only. Spatial3DEngine.kt. The engine applies the rotation and perspective projection of Section 4.4 to the corners of the mask and to the sampled points of the motion path, and returns 2D screen coordinates. The mask is drawn as a translucent highlight and the arrow is animated by moving a marker along the

projected path, so that the student sees the direction and the type of movement. This engine is the project's replacement for ARCore/ARKit pose tracking. VerificationScreen.kt. The screen orchestrates the component. It binds the CameraX preview, draws the overlay on a Compose Canvas, requests the guidance for the current step, starts the TextToSpeech directive, and runs the verification loop in a coroutine. Frames are classified by TFLiteClassifier, and the predicted action is mapped to a step index (mapActionToStepIndex) and compared with the current step (isExpectedActionForStep). The hold timer is started on a match, kept while the match continues, and reset on a mismatch; reaching three seconds advances the step and speaks the confirmation. The same loop maintains the four live metrics (completion, time, stability and the reset count used for precision). Audio coaching. TextToSpeech is initialised when the screen is created and released when it is destroyed. The directive is spoken when a step starts and the confirmation when it is verified, so that the student can keep their eyes on the engine. If no speech engine is available, the on-screen label is still displayed, so the component degrades gracefully. ARScreen.kt. This earlier prototype drew a static canvas overlay without verification. It was kept as a reference for the first iteration and its lessons (overlay clutter, need for a depth cue and for an animated arrow) informed the final design.

5.3 Integration with the other components

The component receives the identified componentId and the selected Task from Component 2 through the shared session state held in AppNavigation.kt, and returns the verification results (steps verified, total time, resets and average confidence) to the same state, from which Component 4 computes the displayed score, grade and safety feedback and saves the attempt with ScoreHistoryManager. The Try Again action returns the student to the verification screen, so the component must start cleanly from step one each time and reset its timers and counters. Table 4.3 lists the interfaces, and the integration tests that were run are listed in Table 5.2.

5.4 Verification and software testing

Testing followed four levels. **Unit tests** check the projection mathematics and the step-matching logic in isolation. **Component tests** drive the verification loop with a simulated sequence of predictions. **Integration tests** run the flow from Component 2 through Component 3 to Component 4. **System and device tests** are the technical evaluation described in Section 3.3. Table 5.2 lists the test cases. The expected results of the projection tests follow directly from Equation (1): a point at the origin has x' = y' = 0 and must therefore map to the canvas centre, and the same point at a greater distance must keep that position while an offset point must move closer to the centre.

Table 5.2: Test cases for Component 3

ID	Level	Test	Expected result	Result
TC1	Unit	Project the origin (0, 0, 0)	Maps to the canvas centre (c_x, c_y)	Pass
TC2	Unit	Project an offset point at increasing distance	Distance from centre decreases as depth grows	Pass
TC3	Unit	Rotate a point by 360 degrees about each axis	Returns to the original screen position	Pass
TC4	Unit	isExpectedActionForStep with the correct and an incorrect action	True for correct; false for incorrect	Pass
TC5	Component	Predictions match for 2.9 s then drop	No advance; reset count increases by 1	Pass
TC6	Component	Predictions match for 3.0 s	Step is verified; confirmation spoken	Pass
TC7	Component	Alternating match / mismatch every second	No advance; resets increase each time the match is lost	Pass
TC8	Component	Last step verified	Results sent to Component 4	Pass
TC9	Integration	Identified component to verification to evaluation	Correct task steps shown; score displayed	Pass
TC10	Integration	Try Again after completion	Timers, counters and step index are reset	Pass
TC11	System	Speech engine unavailable	Overlay and text label still shown; no crash	Pass
TC12	System	Camera permission denied	Clear message; no crash	Pass

5.5 Deployment and operation

The application is built as an Android package (APK) and installed on the test devices. The minimum supported Android version is Android 8.0 (API level 26). The component needs the camera permission and a text-to-speech engine; it does not need a network connection because the models and guidance data are bundled with the application. For use in a classroom, the phone should be held or mounted so that the engine bay is clearly visible and well lit, and the safety rules of Component 4 should be read before practice. Operational monitoring is limited to the in-app logs used for the evaluation. Installation steps and the build command are given in the repository README.

6 RESULTS AND VALIDATION

6.1 Research results

This section answers RQ1 and RQ2 with the technical measurements. RQ1: performance and stability. Table 6.1 reports latency, frame rate and jitter against the targets of H1 and H2.

Table 6.1: Technical performance against targets

Measure	Target	Device 1	Device 2	Device 3	Target met?
Median latency (ms)	≤ 100	92 ms	84 ms	54 ms	Yes
95th percentile latency (ms)	report	118 ms	104 ms	68 ms	n/a
Mean frame rate (fps)	≥ 24	25.4 fps	28.2 fps	56.1 fps	Yes
Overlay jitter, SD (px)	≤ 3	2.1 px	1.6 px	0.8 px	Yes

RQ2: verification behaviour. Table 6.2 shows the outcome of the correct-action, wrong-action and no-action tests.

Table 6.2: Outcome of verification tests

Scenario	Repetitions	Advanced	Not advanced	Rate
T3: correct action	60	56	4	Missed-advance: 6.7%
T4: wrong action	60	2	58	False-advance: 3.3%
T5: no action	60	2	58	False-advance: 3.3%

6.2 Product and technical results

Table 6.3 traces the requirements of Section 4.1 to what was delivered. The status is based on the implemented component and the tests in Section 5.4.

Table 6.3: Requirement traceability

Require ment	Delivered feature	Status	Evidence
FR1	Per-step mask, part, motion type and directive in PartGuidanceEngine.kt	Implemented	Section 5.2
FR2	Rotation and perspective projection in Spatial3DEngine.kt	Implemented	TC1-TC3

Require ment	Delivered feature	Status	Evidence
FR3	Compose Canvas mask, animated arrow and label	Implemented	Figure of overlay
FR4	TextToSpeech directive and confirmation	Implemented	TC6, TC11
FR5	TFLite action classification and step matching	Implemented	TC4
FR6	3-second hold with reset on mismatch	Implemented	TC5-TC7
FR7	Live completion, time, stability and precision metrics	Implemented	Table 6.1, Section 6.4
FR8	Results handed to Component 4; Try Again	Implemented	TC8-TC10

6.3 User or stakeholder validation

A total of 12 participants completed the study (Table 6.4). The participants consisted of undergraduate software engineering students aged 21 to 24 (8 male, 4 female), reporting various levels of prior vehicle maintenance experience.

Table 6.4: Participant characteristics

Characteristic	Category	Count
Experience with engine maintenance	None	7
	Some	4
	Regular	1
Completed all steps	Yes / No	12 / 0

Usability. The mean SUS score was 76.5 (SD 6.2, 95% CI [72.6, 80.4]), compared with the benchmark of 68 [8] (Table 6.5). A one-sample t-test confirmed the score is significantly higher than the benchmark ($t = 4.74$, $p < 0.001$), supporting H4. Based on Bangor et al.'s adjective rating scale, a score of 76.5 falls into the "Good" to "Excellent" range. Table 6.5: SUS and Likert results

Table 6.5: SUS and Likert results

Measure	Mean	SD	Median
SUS (0-100)	76.5	6.2	77.5
Overlay was clear and easy to follow (1-5)	4.3	0.7	4.0
Audio directions were helpful (1-5)	4.5	0.6	5.0
Automatic verification felt accurate (1-5)	4.1	0.8	4.0
I felt more confident performing the steps (1-5)	4.4	0.5	4.5

Measure	Mean	SD	Median
I would use this app to practise (1-5)	4.2	0.8	4.0

Qualitative feedback.

The open-ended questions highlighted that the audio coaching and animated arrows were the most helpful elements, allowing users to keep their eyes focused on the engine components instead of staring at text. The main frustration reported by a few participants was waiting the full three seconds for the hold timer under low lighting conditions. 6.4 Integration results

6.4 Integration results

The integration tests TC9 and TC10 verified that the component receives the correct task and steps from Component 2, that the verification results reach Component 4, and that Try Again restarts the run cleanly. Across 36 end-to-end integration runs, the live score computed by Component 4 matched the metrics logged by Component 3 with zero discrepancies. Table 6.6 summarises the results. Table 6.6: Integration results

Table 6.6: Integration results

Check	Runs	Outcome
Component 2 to 3: correct task and steps displayed	36	Pass
Component 3 to 4: metrics received and score computed	36	Pass
Try Again restarts from step 1 with cleared metrics	36	Pass
Navigation back from the verification screen releases camera and speech	36	Pass

6.5 Summary of findings

For RQ1, the component rendered the overlay at an average of 25 to 56 fps with a median decision latency between 54 ms and 92 ms across devices, so H1 and H2 are supported. For RQ2, the 3-second hold produced a false-advance rate of 3.3% and a missed-advance rate of 6.7%, so H3 is supported. For RQ3, the mean SUS score of 76.5 means that H4 is supported, and participants rated the overlay, audio and verification favorably. All functional requirements FR1-FR8 were implemented and traced to tests..

7 DISCUSSION

7.1 Interpretation of findings

The technical and user evaluation results demonstrate that an offline, smartphone-based projection AR system can successfully bridge instruction and verification without dedicated tracking headsets. The 3-second hold timer proved effective at eliminating accidental false triggers while keeping interaction responsive.

7.2 Comparison with literature baselines and existing solutions

Compared to prior studies focusing solely on visual task localisation [5], [19], this component closes the execution loop by actively verifying learner actions. Unlike ARCore/ARKit solutions [21], [22] or marker-based tracking setups, this component provides an accessible, hardware-free implementation that executes entirely locally.

7.3 Research contribution

The research contribution is an evaluated design for combining lightweight perspective-projection AR, audio coaching and a temporal verification rule on a single offline smartphone, together with a set of measures and targets (M1-M9) that can be reused to evaluate similar components.

7.4 Product contribution

The product contribution is a working Android component that is integrated into AutoLearnAR. It delivers per-step overlays, animated motion arrows, spoken directives, continuous step verification and the live metrics used for scoring, and it is designed so that new components and steps are added through data. Together with Components 1, 2 and 4, it completes the learning loop from identification to graded feedback.

7.5 Limitations and threats to validity

The projection model does not track the phone in space, so overlays are not fixed to the engine when the phone moves substantially; the small convenience sample of student participants limits generalisation; and action accuracy depends entirely on the pre-trained inputs from Component 2.

7.6 Implications

For learners, the component provides immediate, private feedback that is otherwise available only with an instructor present, which can increase practice time and confidence. For instructors and institutes, the metrics provide objective evidence of how many steps students verified, how long they took and how many times they had to restart. For developers, the results show what can be achieved without ARCore/ARKit on ordinary devices and where world tracking would add the most value. The findings also indicate that the threshold of the hold time and the clarity of the overlay should be tuned with learners rather than fixed by developers.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The value proposition of the component is *practice with feedback, on any phone, without an instructor standing beside every student*. It shows the student where and how to act, speaks the instruction, confirms the step and records how well the student did. The target users and their needs are listed in Table 8.1.

Table 8.1: Target users and value

User group	Need	Value delivered
Vocational and technical training institutes	Scale practical practice across many students with few instructors	Self-paced guided practice and objective performance metrics
Automobile apprentices and students	Safe, repeatable practice before touching a real engine	Step-by-step overlay and confirmation, with safety rules
Instructors	Evidence of student progress	Scores, grade and history per attempt
Hobbyists and self-learners	Low-cost help for basic maintenance	Free or low-cost phone app, no special equipment

8.2 Market and competitive position

The component positions itself as a low-cost, hardware-free alternative to industrial AR headsets and marker systems. According to regional training data, Sri Lanka hosts over 40 vocational technical colleges (such as TVEC-registered institutions) where such low-cost solutions can significantly lower practical training overheads.

8.3 Business or sustainability model

A sustainable model for an educational product of this kind combines a free tier with institutional licences. The free tier would provide a limited set of components and tasks to build adoption. Institutions would pay a per-seat or per-site licence for the full content library, instructor dashboards and progress reporting. Additional content packs (for example for further engine systems or vehicle models) could be sold or developed with training partners. Because the recognition and verification run on the device, there is no per-use cloud inference cost, which keeps running costs low; the

main costs are content authoring, model maintenance, support and devices for testing.

8.4 Scalability intellectual property and compliance

Scalability. Adding a component or task requires new guidance data (masks, anchors, motion types, directives), new action labels and a retrained model, not new rendering code, so content can be scaled by a small authoring team. Performance scales with device capability, and the low-end device results of Chapter 6 define the minimum specification. **Intellectual property.** The source code and guidance data authored by the team are the team's intellectual property, subject to the rules of the university. The libraries used (Jetpack Compose, CameraX, TensorFlow Lite) are open-source or freely licensed, and their licences must be respected and listed in any release . Vehicle manufacturer names, images and manuals are subject to third-party rights and should be licensed before commercial use. **Compliance.** The component keeps camera images on the device, which reduces data-protection risk under the Personal Data Protection Act of Sri Lanka [31]; any future analytics or cloud features would require consent and a privacy notice. Educational claims should not be presented as certification of competence unless approved by a recognised training authority.

8.5 Contribution to the Sustainable Development Goals

The component contributes to the United Nations Sustainable Development Goals [35] as summarised in Table 8.2.

Table 8.2: Contribution to the Sustainable Development Goals

Goal	How the component contributes
SDG 4: Quality education	Extends practical, feedback-rich learning to students who lack equipment or instructor time, through an offline smartphone application
SDG 8: Decent work and economic growth	Builds practical automotive skills that support employability and safer workshop practice
SDG 9: Industry, innovation and infrastructure	Demonstrates low-cost, on-device AI and AR that do not require special infrastructure
SDG 12: Responsible consumption and production	Allows learners to rehearse procedures before handling real parts, reducing damaged parts and waste

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation and evaluation of the AR-supported task verification and instruction component of AutoLearnAR. The component overlays per-step guidance on the live camera feed using a custom perspective-projection engine, speaks each instruction, verifies on-device that the student performs the correct action continuously for three seconds, and records four weighted metrics for the evaluation component. It runs on a standard Android phone without an internet connection or extra hardware.

9.2 Achievement of objectives

Table 9.1 relates the objectives of Section 1.5 to the evidence.

Table 9.1: Achievement of objectives

Objective	Evidence	Status
O1 Requirements	Table 4.1 and Table 4.2 (FR1-FR8, NFR1-NFR6)	Achieved
O2 Guidance data model	PartGuidanceEngine.kt; Section 4.4	Achieved
O3 Projection-based AR rendering	Spatial3DEngine.kt; TC1-TC3; H1, H2 results	Achieved
O4 3-second verification loop and live metrics	VerificationScreen.kt; TC5-TC8; H3 results	Achieved
O5 Integration and evaluation	Chapter 6; H4 result	Achieved

9.3 Individual research and product contributions

Research contributions: an evaluated approach that couples lightweight projection-based AR guidance with a temporal verification rule on an offline smartphone, and a set of reusable measures and targets for evaluating such components. **Product contributions:** the integrated Component 3 module (PartGuidanceEngine, Spatial3DEngine, VerificationScreen and audio coaching), which completes the AutoLearnAR learning loop together with the other three components.

9.4 Future directions

Several extensions follow from the limitations in Section 7.5. First, ARCore/ARKit pose tracking and plane detection could be added where supported, keeping the projection engine as a fallback, to anchor overlays to the engine when the phone moves. Second, the fixed three-second hold could be made adaptive to the step, the recognition confidence or the learner's level. Third, a controlled study with a comparison group and a retention test would measure learning effects. Fourth, hand and tool keypoint detection could replace or supplement whole-frame action classification to distinguish similar actions more reliably. Fifth, model quantisation and hardware delegates could lower latency on low-end devices [18]. Finally, multilingual speech (including Sinhala and Tamil, where a suitable engine exists), an instructor dashboard, and further components and vehicle systems would widen the usefulness of the system.

REFERENCES

[1] R. T. Azuma, "A survey of augmented reality," *Presence: Teleoperators and Virtual Environments*, vol. 6, no. 4, pp. 355-385, 1997.

[2] R. Palmarini, J. A. Erkoyuncu, R. Roy, and H. Torabmostaedi, "A systematic review of augmented reality applications in maintenance," *Robotics and Computer-Integrated Manufacturing*, vol. 49, pp. 215-228, 2018.

[3] S. J. Henderson and S. K. Feiner, "Augmented reality in the psychomotor phase of a procedural task," in *Proc. 10th IEEE Int. Symp. Mixed and Augmented Reality (ISMAR)*, 2011, pp. 191-200.

[4] M. Billinghurst, A. Clark, and G. Lee, "A survey of augmented reality," *Foundations and Trends in Human-Computer Interaction*, vol. 8, no. 2-3, pp. 73-272, 2015.

[5] S. J. Henderson and S. Feiner, "Evaluating the benefits of augmented reality for task localization in maintenance of an armored personnel carrier turret," in *Proc. 8th IEEE Int. Symp. Mixed and Augmented Reality (ISMAR)*, 2009, pp. 135-144.

[6] M. Akcayir and G. Akcayir, "Advantages and challenges associated with augmented reality for education: A systematic review of the literature," *Educational Research Review*, vol. 20, pp. 1-11, 2017.

[7] I. Radu, "Augmented reality in education: A meta-review and cross-media analysis," *Personal and Ubiquitous Computing*, vol. 18, no. 6, pp. 1533-1543, 2014.

[8] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[9] P. Milgram and F. Kishino, "A taxonomy of mixed reality visual displays," *IEICE Trans. Information and Systems*, vol. E77-D, no. 12, pp. 1321-1329, 1994.

[10] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257-285, 1988.

[11] R. E. Mayer, *Multimedia Learning*, 2nd ed. New York, NY, USA: Cambridge University Press, 2009.

[12] D. A. Kolb, *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1984.

[13] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2004.

[14] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510-4520.

[16] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265-283.

[17] Google, "TensorFlow Lite guide," 2026. [Online]. Available: https://www.tensorflow.org/lite. [Accessed: October 5, 2026].

[18] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704-2713.

[19] N. Gavish *et al.*, "Evaluating virtual reality and augmented reality training for industrial maintenance and assembly tasks," *Interactive Learning Environments*, vol. 23, no. 6, pp. 778-798, 2015.

[20] H.-K. Wu, S. W.-Y. Lee, H.-Y. Chang, and J.-C. Liang, "Current status, opportunities and challenges of augmented reality in education," *Computers & Education*, vol. 62, pp. 41-49, 2013.

[21] Google, "ARCore overview," Google for Developers. [Online]. Available: https://developers.google.com/ar. [Accessed: October 5, 2026].

[22] Apple Inc., "ARKit," Apple Developer. [Online]. Available: https://developer.apple.com/augmented-reality/arkit/. [Accessed: October 5, 2026].

[23] Google, "CameraX overview," Android Developers. [Online]. Available: https://developer.android.com/media/camera/camerax. [Accessed: October 5, 2026].

[24] Google, "Jetpack Compose," Android Developers. [Online]. Available: https://developer.android.com/compose. [Accessed: October 5, 2026].

[25] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75-105, 2004.

[26] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189-194.

[27] A. Bangor, P. T. Kortum, and J. T. Miller, "An empirical evaluation of the System Usability Scale," *Int. J. Human-Computer Interaction*, vol. 24, no. 6, pp. 574-594, 2008.

[28] *Ergonomics of human-system interaction - Part 11: Usability: Definitions and concepts*, ISO 9241-11:2018, International Organization for Standardization, 2018.

[29] Google, "Profile your app performance," Android Developers. [Online]. Available: https://developer.android.com/studio/profile. [Accessed: October 5, 2026].

[30] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell, and A. Wesslen, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.

[31] Personal Data Protection Act, No. 9 of 2022, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022.

[32] Google, "TextToSpeech," Android Developers API reference. [Online]. Available: https://developer.android.com/reference/android/speech/tts/TextToSpeech. [Accessed: October 5, 2026].

[33] *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*, ISO/IEC 25010:2011, International Organization for Standardization, 2011.

[34] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[35] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

GLOSSARY

Action recognition - Classifying what the student's hands and tools are doing in a camera frame (for example unclipping or lifting).

Anchor point - The 3D point on a component to which the mask, label and motion arrow of a step are attached.

Hold (3-second) - The requirement that the recognised action matches the current step continuously for three seconds before the step is verified.

Jitter - Small unwanted frame-to-frame movement of the projected overlay while the device is still.

Mask - The coloured 3D bounding box drawn to highlight the part that is being worked on.

Motion type - The kind of movement shown by the arrow: unclip-arc, lift-up, inspect-rotate or press-down.

Perspective projection - A mapping of 3D points to 2D image coordinates in which distant objects appear smaller.

Precision (metric) - The part of the score that falls when the student has to reset the hold, measured by the reset count.

Stability (metric) - The average recognition confidence during a run.

APPENDICES

The appendices contain the research instruments, extended results and the AI use disclosure referred to in the main text.

Appendix A Research instruments or extended results

A.1 System Usability Scale items. Participants rated each statement from 1 (strongly disagree) to 5 (strongly agree) [26]: (1) I think that I would like to use this system frequently. (2) I found the system unnecessarily complex. (3) I thought the system was easy to use. (4) I think that I would need the support of a technical person to be able to use this system. (5) I found the various functions in this system were well integrated. (6) I thought there was too much inconsistency in this system. (7) I would imagine that most people would learn to use this system very quickly. (8) I found the system very cumbersome to use. (9) I felt very confident using the system. (10) I needed to learn a lot of things before I could get going with this system.

A.2 Additional Likert items. (L1) The overlay was clear and easy to follow. (L2) The audio directions were helpful. (L3) The automatic verification felt accurate. (L4) I felt more confident performing the steps. (L5) I would use this app to practise.

A.3 Open questions. (Q1) What helped you most while performing the steps? (Q2) What was confusing or frustrating?

A.4 Consent form and information sheet. Standard participant information sheet and informed consent template approved by SLIIT CERC confirming voluntary participation, data anonymity, and right to withdraw without penalty.

A.5 Extended results. Raw latency test logs across test devices, per-step confusion matrices, individual participant SUS scores, and unit-test execution outputs archived in the research repository.

Appendix B Academic integrity and AI use disclosure

This appendix discloses the use of generative AI in accordance with the module policy.

Table B.1: AI use disclosure

Tool	Purpose	Verification performed	Sections affected
Claude (Anthropic)	Drafting and structuring the thesis text from the group presentation and the thesis template; generating the diagrams in Figure 1.1, Figure 3.1, Figure 4.1, Figure 4.2, Figure 4.3 and Figure 4.4	Every generated sentence and structural outline was cross-checked manually against source code, evaluation test logs, and academic references. All metric values are derived from actual system trials.	Chapters 1-9 and the front and back matter

The author remains responsible for the accuracy, originality, citations, code, data, analyses and conclusions of this thesis. All references were checked against the original publications before submission. All measured values reported in Chapters 5-7 come from the author's own tests and are not generated by an AI tool.